Lightweight Anomaly Detection for Network Telemetry

Using MAD and Kalman Filter

A practical approach to detecting anomalies in network switch telemetry data using lightweight algorithms suitable for resource-constrained devices.

What is Network Telemetry?

Modern network devices continuously export metrics at regular intervals, creating time-series data streams that help monitor network health and performance.

Packets per Second
Throughput measurement

Queue Depth
Buffer occupancy

Packet Drops
Loss events

CPU Utilisation
Processing load

Interface Errors
Transmission issues

Sample Telemetry Data

Time	Packets/Sec
T1	1000
T2	1020
T3	1010
T4	1030
T5	**5000**
T6	1040

The value **5000** represents a spike anomaly that needs detection.

Problem Statement

The Challenge

Telemetry signals present unique challenges for anomaly detection:

- Noisy measurements with random fluctuations
- Constantly changing baseline values
- Occasional spikes and sudden changes
- Resource constraints on network devices

Our Goal

Detect meaningful anomalies such as traffic spikes, congestion events, and sudden metric changes whilst filtering out normal noise.

Critical requirement: Algorithms must be lightweight enough to run on network devices with limited CPU and memory resources.

MAD: Core Idea

Median Absolute Deviation

MAD measures the typical variability of a signal in a robust way that's resistant to outliers.

01

Compute Median

Find the median of values in the sliding window

02

Calculate Deviations

Compute absolute deviation of each value from median

03

Find MAD

MAD = median of all absolute deviations

Formula: $MAD = \text{median}(|x_i - \text{median}(x)|)$

MAD Example with Telemetry

Working Through the Math

Consider a sliding window of packet rates: $[1000, 1020, 980, 1010, 995]$

1

Step 1: Median

Sorted: [980, 995, 1000, 1010, 1020]

Median = 1000

2

Step 2: Deviations

|1000–1000| = 0

|1020–1000| = 20

|980–1000| = 20

|1010–1000| = 10

|995–1000| = 5

3

Step 3: MAD

Sorted deviations: [0, 5, 10, 20, 20]

MAD ≈ 10

Detecting the Anomaly

New value arrives: **5000**

MAD score = |5000 – 1000| / 10 = **400**

A large MAD score indicates an anomaly!

MAD: Strengths and Limitations

Strengths

- **Robust to outliers** - median-based calculation ignores extreme values
- **Simple computation** - minimal CPU overhead
- **Small windows work well** - adapts quickly to changes
- **Good for spike detection** - identifies sudden deviations effectively

Limitations

- **Requires sliding window** - needs to maintain recent values in memory
- **Cannot detect gradual drift** - slow changes may be mistaken for normal variation
- **Fixed threshold needed** - requires tuning for different metrics

What Kalman Filter Really Does

Kalman is primarily a **dynamic baseline estimator**. It predicts the next value of the signal using the previous estimate and the current observation.

01

State Prediction

Predict the next value based on previous estimate

02

Measurement Update

Incorporate new observation with Kalman gain

03

Baseline Output

Produce smoothed baseline estimate

The Update Formula

new_estimate = prediction + K × (measurement − prediction)

Where K = Kalman gain (0–1, we trust previous estimate more than new measurement)

Key Point: Kalman produces baseline estimates but does <u>not</u> directly decide if something is anomalous

Step-by-Step: Initial Values

At t=1

Step	Value
Observed	1000
Estimate	1000
Residual	0

Initial observation becomes the first estimate

At t=2

Step	Value
Prediction	1000
Measurement	1020
Residual	20
Update	1004

Estimate = $1000 + 0.2 \times 20 = 1004$

Step-by-Step: Building the Baseline

t=3

Prediction: 1004

Measurement: 1010

Residual: 6

Estimate: 1005.2

1

2

t=4

Prediction: 1005.2

Measurement: 1030

Residual: 24.8

Estimate: ~1010

The baseline gradually adapts to the signal's typical values

The Spike: Where Kalman Shows Its Value

At t=5: The Spike

Component	Value
Observed	5000
Prediction	1010
Residual	3990
Estimate	1818

Estimate = 1010 + 0.2 × 3990 = 1818

Why This Matters

The estimate **does not jump to 5000** — the filter dampens the spike

This is why Kalman filters are excellent for **noise filtering**

Expected baseline remains ~1000, not 5000

Why Kalman + MAD Works Better Together

Case 1: Slow Trend

Telemetry: 1000 1100 1200 1300 1400 1500

MAD alone: May think this is normal (gradual increase)

Kalman: Tracks the trend, baseline increases appropriately

Case 2: Noisy Signals

Telemetry: 1000 980 1040 970 1020 995

MAD alone: May fluctuate with window changes

Kalman: Smooths signal, provides stable baseline

Case 3: Spike Detection

Telemetry: 1000 1020 1010 1030 5000 1040

Kalman: Baseline ~1010

Deviation: |5000 – 1010| = 3990

MAD: Evaluates if this deviation is abnormal

The Real Pipeline

Raw Telemetry

1000 1020 1010 1030 5000 1040

Kalman Filter

Produces baseline estimate ~1010

Calculate Residuals

10 5 20 3990 15

Apply MAD

Detects extreme residuals

Generate Alert

Anomaly detected!

Key Insight: MAD runs on residual signal, not raw telemetry

Why This Reduces False Positives

Without Kalman

MAD runs directly on telemetry:

- Traffic increases gradually
- MAD may falsely flag anomalies
- Baseline doesn't adapt to trends

With Kalman

MAD runs on residuals:

- Baseline adapts to trend
- MAD sees only unexpected deviations
- Accurate anomaly signals